

SMART INDIA HACKATHON 2026

Technical Project Report

AI-Assisted Virtual Camera Tracking System for Coarse Alignment of Mobile FSOC Terminals

Problem Statement Code	SIH26169
Problem Statement	Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals
Organization	Indian Space Research Organisation (ISRO)
Department	Department of Space / Indian Space Research Organisation
Category	Software
Team Name	404 Not Found
Team ID	155387
Institute	Tecnia Institute of Advanced Studies

Submission Document

Declaration

This technical report describes the software prototype submitted by Team 404 Not Found for Smart India Hackathon 2026 Problem Statement SIH26169. The document is based on the implementation contained in the submitted source repository.

The report deliberately distinguishes implemented capabilities from proposed future enhancements. In particular, the present prototype uses deterministic computer-vision, state-estimation, search, and feedback-control methods; it does not contain a trained deep-learning model. The SIH problem statement describes an AI-assisted solution, and the architecture is intentionally modular so a learned detector or adaptive controller can be introduced without replacing the virtual camera, disturbance, logging, and control infrastructure.

Executive Summary

Free Space Optical Communication (FSOC) uses highly directional optical beams. Before a fine-pointing stage can establish precise alignment, a coarse Pointing, Acquisition and Tracking (PAT) stage must locate the remote optical beacon and continuously keep it inside a camera field of view. Performing this development directly on physical cameras, optical benches, pan-tilt mechanisms, and communication terminals is costly and difficult to reproduce.

This project provides a software-only coarse-alignment laboratory. A configurable 2000×2000 virtual scene contains a moving optical beacon. A virtual 640×480 monochrome camera extracts a field of view from that scene. An Archimedean spiral scanner performs acquisition when the target is not visible. Once the beacon is detected, a robust image-processing detector estimates its centroid. A two-dimensional constant-velocity Kalman filter smooths the measurement and supplies prediction during short visual dropouts. A PID controller converts image-plane error into virtual pan/tilt motion. The loop is closed continuously, while disturbances such as haze, fog, rain, low light, sensor noise, camera jitter, and platform drift can be injected.

The desktop dashboard exposes live camera imagery, world overview, operating state, tracking error, centroid RMSE, lock retention, acquisition time, processing rate, disturbance settings, target motion, presets, reset/pause controls, video input, and benchmark export. The logging subsystem writes frame-level telemetry to CSV and a session summary to JSON.

An included headless edge-case acquisition test was executed against a target initialized near a difficult search position. The test acquired the target in 9 frames, corresponding to approximately 0.30 seconds at the nominal 30 FPS simulation rate, and reported PASS against the 2-second acquisition criterion. Full-session tracking metrics are generated by the application at runtime rather than being fabricated in this report.

Contents

1	Problem Understanding	1
1.1	FSOC and the PAT Problem	1
1.2	SIH Functional Objective	1
2	Objectives	2
3	System Overview	2

3.1	High-Level Architecture	2
3.2	Software Components	3
4	Virtual Environment and Camera Model	3
4.1	Scene	3
4.2	Viewport Extraction	3
5	Target Motion Models	3
5.1	Horizontal Motion	4
5.2	Straight-Line Motion	4
5.3	Circular Motion	4
5.4	Figure-of-Eight Motion	4
5.5	Random Walk	4
6	Acquisition Strategy	4
6.1	Why Search Is Required	4
6.2	Archimedean Spiral Search	4
6.3	Acquisition Criterion	5
7	Beacon Detection	5
7.1	Preprocessing	5
7.2	Morphological Top-Hat Enhancement	5
7.3	Adaptive Thresholding	5
7.4	Candidate Filtering	5
8	State Estimation	6
8.1	Constant-Velocity Kalman Filter	6
8.2	Short-Dropout Handling	6
9	Closed-Loop Pan/Tilt Control	6
9.1	Image-Plane Error	6
9.2	PID Controller	6
10	Disturbance and Robustness Model	7
10.1	Atmospheric Effects	7
10.2	Sensor Noise	7
10.3	Camera Jitter	7
10.4	Platform Drift	7
10.5	Why Disturbance Injection Matters	8

11 Operating State Machine	8
12 Desktop Dashboard	8
12.1 Camera Viewport	8
12.2 World Overview	8
12.3 Live Telemetry	8
12.4 Configuration Controls	9
12.5 Presets	9
12.6 Actions	9
13 Performance Measurement	9
13.1 RMSE	10
13.2 Lock Retention	10
13.3 Frame Rate	10
14 Verification and Test Methodology	10
14.1 Automated Edge-Case Acquisition Test	10
14.2 Functional Test Matrix	11
14.3 Interpretation of Results	11
15 Implementation Technologies	12
16 AI and Algorithmic Positioning	12
16.1 Current Implementation	12
16.2 Why This Is Deliberately Documented	12
16.3 AI Extension Path	13
17 Limitations	13
18 Future Enhancements	13
19 Reproducibility and Deployment	14
19.1 Environment	14
19.2 Installation	14
19.3 Execution	14
19.4 Output Files	14
20 Deliverable Readiness	14
21 Conclusion	15

A Repository File Reference	15
B Key Parameters	16

1 Problem Understanding

1.1 FSOC and the PAT Problem

Free Space Optical Communication transmits information through a highly directional optical path. Unlike a broad radio-frequency coverage pattern, an optical link can be disrupted by a comparatively small pointing error. For mobile terminals, the relative direction between transmitter and receiver changes continuously because of platform motion, target motion, vibration, and environmental effects.

A practical PAT chain is commonly separated conceptually into:

1. coarse acquisition and alignment;
2. fine pointing and alignment; and
3. communication/link maintenance.

This project addresses the software simulation of the first stage. Its purpose is not to claim a complete optical communication terminal, but to provide a repeatable environment in which coarse acquisition and tracking logic can be developed and stressed without specialized optical hardware.

1.2 SIH Functional Objective

The problem statement asks for software capable of autonomously detecting, identifying, and continuously tracking a designated moving target within a virtual scene by controlling a virtual camera viewport. The expected solution includes a configurable virtual environment, moving target generation, a movable camera, automatic beacon detection, continuous computer-vision tracking, camera repositioning, disturbance injection, and real-time performance statistics.

The submitted implementation maps these requirements as shown in Table 1.

Table 1: Requirement-to-implementation mapping

Requirement	Status	Implementation evidence
Virtual environment	Implemented	2000×2000 grayscale scene generated in <code>step2_camera.py</code> .
Moving target	Implemented	Horizontal, straight-line, circular, figure-of-eight, and random motion models.
Movable virtual camera	Implemented	640×480 viewport with configurable FOV and virtual pan/tilt position.
Automatic detection	Implemented	Morphological top-hat enhancement, thresholding, contour filtering, centroid extraction.
Continuous tracking	Implemented	Kalman state estimation and closed-loop PID correction.
Acquisition/search	Implemented	FOV-aware Archimedean spiral scanner.
Disturbances	Implemented	Haze, fog, rain, low-light, salt-and-pepper, Gaussian, Poisson, jitter, and platform drift.

Requirement	Status	Implementation evidence
Real-time statistics	Implemented	GUI telemetry for state, error, RMSE, lock rate, acquisition time, FPS and latency.
Performance log	Implemented	Streaming CSV telemetry and summary JSON report.
Video input	Implemented	MP4/video-file source through OpenCV.
Standalone executable	Not included in repository	The source is runnable as a Python desktop application; packaging with PyInstaller is a deployment step recommended for the final executable deliverable.
Learned AI model	Not included in current source	The current core is deterministic computer vision plus estimation/control; a learned detector/controller is a future extension.

2 Objectives

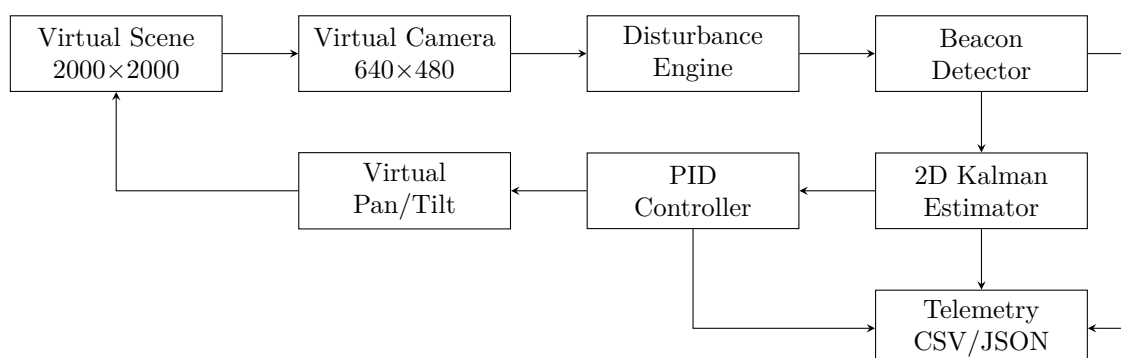
The project objectives are:

1. create a reproducible software environment for coarse FSOC alignment;
2. simulate a moving optical beacon and a steerable camera;
3. acquire a target from an unknown initial camera position;
4. detect the beacon under image degradation;
5. estimate target position and velocity robustly;
6. close the feedback loop through virtual pan/tilt control;
7. tolerate short target-detection losses through prediction and controlled re-search;
8. expose meaningful performance criteria during operation; and
9. automatically preserve telemetry and session-level benchmark information.

3 System Overview

3.1 High-Level Architecture

The system follows a closed-loop pipeline:



3.2 Software Components

The repository contains the following principal modules:

- dashboard.py** Main desktop Tkinter application. Integrates the simulation source, detector, Kalman filter, PID controller, scanner, telemetry logger, GUI, presets, video loading, and report export.
- step2_camera.py** Virtual scene and camera model. Generates the beacon, target trajectories, camera crop, FOV behavior, optical disturbances, sensor noise, jitter, and carrier drift.
- step4_search.py** Archimedean spiral search implementation and a binary target-visibility helper.
- step5_tracker.py** Standalone/reference tracking implementation with an OpenCV control panel and tracking loop.
- step4_dashboard.py** Earlier acquisition-stage dashboard retained as a reference implementation.
- step4_edge_test.py** Headless acquisition test for an edge-case target start position.
- logger.py** Streaming performance logger producing CSV frame telemetry and JSON summary reports.

4 Virtual Environment and Camera Model

4.1 Scene

The simulated world is a 2000×2000 pixel monochrome environment. The beacon is represented as a small bright rectangular target with a default target size of 10 pixels, within the stated 5–20 pixel design range.

The camera begins at the centre of the scene. The camera viewport is 640×480 pixels. At the reference field of view of 4 degrees, the horizontal image scale is:

$$\text{pixels per degree} = \frac{640}{4} = 160 \text{ pixels/degree.}$$

The camera FOV can be changed in the desktop interface. A smaller FOV produces a narrower angular view and therefore changes the relationship between camera angular velocity and pixel displacement.

4.2 Viewport Extraction

The camera source first constructs the complete scene, computes the crop dimensions from FOV, extracts the camera-centred region, pads regions that extend outside the world boundary, and resizes the result to 640×480 .

This approach provides a deterministic mapping between world coordinates and image coordinates while keeping the implementation computationally lightweight.

5 Target Motion Models

Five target motion patterns are available.

5.1 Horizontal Motion

The target moves along the horizontal axis with a nominal velocity of 3.2 pixels per simulation frame and reverses direction at a defined scene margin.

5.2 Straight-Line Motion

The target uses independent horizontal and vertical velocity components. Boundary contact reverses the corresponding component, producing a bounded diagonal trajectory.

5.3 Circular Motion

The target follows:

$$x = x_c + r \cos \theta, \quad y = y_c + r \sin \theta,$$

with an angular increment per frame.

5.4 Figure-of-Eight Motion

The target follows a lemniscate-like path:

$$x = x_c + a \sin \theta, \quad y = y_c + b \sin(2\theta).$$

5.5 Random Walk

The target velocity is perturbed by random increments and constrained by a maximum speed. Boundary conditions reverse velocity components to retain the target inside the scene.

These trajectories are useful because a tracker that only performs well on a predictable horizontal path may fail on changing curvature or irregular motion.

6 Acquisition Strategy

6.1 Why Search Is Required

When the target is outside the current camera FOV, a tracker cannot compute a meaningful image-plane error. The system therefore separates operation into acquisition and tracking phases.

6.2 Archimedean Spiral Search

The search module uses an Archimedean spiral:

$$r = b\theta.$$

The spiral pitch is derived from the camera viewport height and an overlap factor:

$$p = H(1 - \eta),$$

where $H = 480$ pixels and $\eta = 0.30$, giving a pitch of approximately 336 pixels per revolution.

Since:

$$b = \frac{p}{2\pi},$$

the next angular increment is selected from the desired maximum linear pixel speed:

$$\Delta\theta = \frac{v_{\max}}{\sqrt{b^2 + r^2}}.$$

This produces a continuous search path whose instantaneous motion is constrained by the configured camera speed.

6.3 Acquisition Criterion

The main dashboard considers a target acquired when the estimated target position is within 10 pixels of the optical setpoint at the centre of the 640×480 viewport. The nominal acquisition requirement implemented in the code is:

$$t_{\text{acq}} \leq 2.0 \text{ s.}$$

The included edge-case test searches for a target initialized at (640, 708). The executed result was 9 frames at a nominal 30 FPS, or:

$$t = \frac{9}{30} = 0.30 \text{ s,}$$

which passes the 2-second criterion.

7 Beacon Detection

7.1 Preprocessing

The detector first applies a 5×5 median filter. This reduces isolated pixel noise while preserving the compact beacon structure.

7.2 Morphological Top-Hat Enhancement

A 21×21 rectangular top-hat operation is applied to enhance bright structures relative to their local background. This is particularly useful for a beacon represented as a small bright object against a dark scene.

7.3 Adaptive Thresholding

The implementation examines the maximum response of the top-hat image. If the response is weak, a high-percentile threshold of the filtered frame is used. Otherwise, the threshold is selected as a fraction of the maximum top-hat response with a lower bound.

7.4 Candidate Filtering

Morphological opening removes small isolated structures. External contours are extracted. Candidate contours are retained only when their area is between the configured minimum and maximum. Image moments then provide the centroid:

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}}.$$

The candidate whose area is closest to the nominal 100-pixel beacon area is selected.

8 State Estimation

8.1 Constant-Velocity Kalman Filter

The estimator uses the state:

$$\mathbf{x} = [x, y, v_x, v_y]^T.$$

With timestep Δt , the state transition is:

$$\mathbf{x}_{k+1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_k + \mathbf{w}_k.$$

The measurement contains image coordinates:

$$\mathbf{z}_k = [x_m, y_m]^T.$$

The filter predicts the target before measurement update and corrects the estimate using the observed centroid. This reduces the effect of noisy centroid measurements and provides a predicted position during short detection losses.

8.2 Short-Dropout Handling

The main tracking loop allows a limited number of lost frames. For the configured loss limit of 15 frames, the controller can continue using the predicted Kalman position. At 30 FPS, 15 frames correspond to approximately 0.5 seconds. Beyond this limit the system resets the control/search process and returns to acquisition.

This behavior is intentionally conservative: prolonged absence of a reliable beacon should trigger re-acquisition rather than uncontrolled extrapolation.

9 Closed-Loop Pan/Tilt Control

9.1 Image-Plane Error

The desired optical centre is:

$$(x_s, y_s) = (320, 240).$$

The Kalman-estimated target position (x_k, y_k) gives:

$$e_x = x_k - 320, \quad e_y = y_k - 240.$$

The scalar tracking error used for benchmark evaluation is:

$$e = \sqrt{e_x^2 + e_y^2}.$$

9.2 PID Controller

For each axis, the controller computes:

$$u = K_p e + K_i \sum e + K_d \Delta e.$$

The implemented default gains are:

$$K_p = 0.58, \quad K_i = 0.035, \quad K_d = 0.11.$$

The command magnitude is limited to the configured maximum camera speed. When the requested vector exceeds the maximum, both components are scaled proportionally. This avoids commanding a virtual camera beyond its configured velocity envelope.

10 Disturbance and Robustness Model

The simulator deliberately separates environmental effects from the tracking algorithm.

10.1 Atmospheric Effects

The supported modes are:

- clear;
- haze;
- fog;
- rain; and
- low light.

The effects are implemented as image transformations. Haze and fog reduce contrast and add a brightness component. Low light reduces intensity. Rain introduces synthetic streaks while changing overall intensity.

10.2 Sensor Noise

The supported sensor noise modes are:

- none;
- salt-and-pepper;
- Gaussian; and
- Poisson.

The GUI exposes a noise level from 0 to 20. Salt-and-pepper noise interprets the level as a percentage-like probability, while Gaussian and Poisson modes use the configured level to control their respective distributions.

10.3 Camera Jitter

Camera jitter is injected as independent random pixel displacement in the horizontal and vertical directions. The configured range is 0–20 pixels per frame.

10.4 Platform Drift

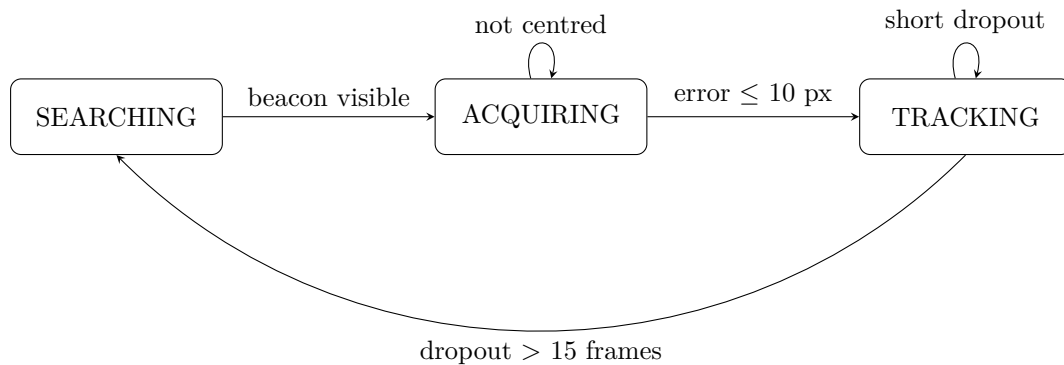
Platform drift is represented by a time-varying sinusoidal displacement in both axes. This approximates low-frequency carrier/platform motion rather than high-frequency sensor noise.

10.5 Why Disturbance Injection Matters

A tracking algorithm should not be evaluated only in a clean scene. The disturbance engine makes it possible to repeat experiments with controlled stress levels and to compare detector and controller behavior under progressively degraded observations.

11 Operating State Machine

The main dashboard uses the following conceptual states:



In implementation terms, the system begins in **SEARCHING**. Once a beacon measurement exists, the system can enter **ACQUIRING**. When the Kalman-estimated centroid reaches the 10-pixel lock criterion, the state becomes **TRACKING**. Short losses are handled with prediction; prolonged loss returns the system to search.

12 Desktop Dashboard

The primary application is `dashboard.py`, a Tkinter desktop interface.

12.1 Camera Viewport

The left display shows the current 640×480 virtual camera feed. The optical centre is marked and the detected/predicted beacon location is overlaid.

12.2 World Overview

The second display shows the 2000×2000 world at reduced scale. It provides spatial context for the target, camera FOV footprint, and virtual pan/tilt position.

12.3 Live Telemetry

The telemetry area reports:

- operating state;
- pan/tilt angle;
- tracking error;

- centroid RMSE;
- lock retention rate;
- acquisition time; and
- processing/FPS information.

12.4 Configuration Controls

The control panel exposes:

- target motion;
- noise type and level;
- atmosphere type and severity;
- camera jitter;
- platform drift; and
- pan speed.

12.5 Presets

Four quick presets are provided:

1. Clear Sky;
2. 10% Salt-and-Pepper Noise;
3. Heavy Fog (75%);
4. High Jitter (± 20 pixels).

12.6 Actions

The dashboard provides pause/resume, simulation reset, MP4/video loading, and benchmark report export.

13 Performance Measurement

The performance logger writes one CSV row per processed frame. The recorded fields include:

- timestamp;
- frame ID;
- operating state;
- target motion;
- target position;
- camera position;
- pan/tilt angles;
- raw/Kalman tracking error;

- FPS;
- processing latency;
- noise mode; and
- atmosphere mode.

The session JSON report computes acquisition time, average reacquisition time when available, RMSE, average error, maximum error, lock retention rate, target-loss/out-of-spec proportion, average FPS, and average processing latency.

13.1 RMSE

For tracked error samples e_i :

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}.$$

The implementation checks the tracking error against 10 pixels:

$$RMSE \leq 10 \text{ px.}$$

13.2 Lock Retention

During tracked frames, the logger counts frames with:

$$e_i \leq 10 \text{ px.}$$

The lock-retention rate is:

$$L = \frac{N_{\text{in-spec}}}{N_{\text{tracked}}} \times 100.$$

The dashboard uses 95% as its displayed lock-retention target.

13.3 Frame Rate

The logger computes average session FPS as:

$$FPS_{\text{avg}} = \frac{N_{\text{frames}}}{T}.$$

The implementation treats 20 FPS as the displayed minimum performance criterion.

14 Verification and Test Methodology

Testing is divided into functional, acquisition, disturbance, tracking, and logging checks.

14.1 Automated Edge-Case Acquisition Test

The supplied `step4_edge_test.py` starts the beacon at:

$$(640, 708)$$

with the nominal 4-degree FOV, 8 degree/s maximum pan speed, and 30 FPS.

The executed result was:

- acquisition: 9 frames;
- nominal time: approximately 0.30 s;
- requirement: ≤ 2.0 s;
- result: PASS.

This confirms that the search module can acquire the selected edge-case target well within the implemented acquisition threshold.

14.2 Functional Test Matrix

Table 2: Recommended submission validation matrix

Test	Procedure	Expected result
Clean acquisition	Clear environment, default target	Beacon acquired and enters tracking.
Motion diversity	Run all five motion modes	Target remains detectable and controller responds.
Salt-and-pepper	Increase noise toward 10%	Detector remains functional for supported conditions.
Gaussian noise	Increase noise level	Tracking error and lock statistics remain observable.
Fog	Set heavy fog preset	Reduced contrast is visible and tracker attempts continued lock.
Low light	Increase low-light severity	Detector operates under reduced intensity within supported range.
Jitter	Set ± 20 px jitter	Kalman filtering reduces measurement variation and controller responds.
Platform drift	Increase drift	Camera compensation follows moving target/scene relationship.
Dropout	Temporarily remove target visibility	Prediction/coasting is attempted before re-search.
Logging	Run session and export report	CSV and JSON files are generated in <code>logs/</code> .
Video source	Load supported video file	Video frames are processed through the dashboard source path.
Reset	Change parameters and press reset	Simulation returns to initial state.

14.3 Interpretation of Results

The software is designed to generate the definitive session metrics itself. This report does not invent a single universal RMSE, FPS, or lock rate because those values depend on target trajectory, disturbance settings, host machine, and session duration. The correct submission evidence is the exported JSON/CSV report from the actual demonstration run.

15 Implementation Technologies

The project is implemented in Python with:

- OpenCV for image processing, video handling, morphology, thresholding, contours, and display;
- NumPy for arrays, numerical calculations, random disturbance generation, and Kalman matrix operations;
- Pillow for conversion of OpenCV frames into Tkinter-compatible images;
- Tkinter for the desktop dashboard;
- Python standard library modules for logging, file handling, timing, CSV, and JSON.

The supplied `requirements.txt` contains only:

```
opencv-python
numpy
Pillow
```

No database, network service, external API, cloud service, or GPU-specific runtime is required by the submitted source.

16 AI and Algorithmic Positioning

16.1 Current Implementation

The SIH problem statement uses the term AI-assisted. The submitted prototype currently implements a deterministic algorithmic stack rather than a trained neural network. Its intelligence comes from:

1. adaptive visual preprocessing;
2. statistical/morphological target selection;
3. Kalman state estimation and prediction;
4. closed-loop feedback control; and
5. adaptive acquisition/re-acquisition logic.

This choice reduces installation complexity and makes the prototype deterministic, inspectable, and suitable for a software-only coarse-alignment demonstrator.

16.2 Why This Is Deliberately Documented

Claiming a CNN, YOLO model, transformer, reinforcement-learning agent, or trained classifier when none exists in the submitted repository would make the report technically inaccurate. The present architecture instead provides clear insertion points for a learned detector or adaptive controller.

16.3 AI Extension Path

A future learned detector can replace or augment `RobustBeaconDetector`. A model can output a beacon bounding box or centre, after which the same Kalman and PID layers can be retained. A learned policy or adaptive gain scheduler could also tune control parameters according to estimated disturbance severity.

17 Limitations

The following limitations are explicitly acknowledged:

1. The current environment is two-dimensional rather than a full 3D optical simulation.
2. The camera model is a simplified viewport crop rather than a physical lens, sensor, and optical propagation model.
3. The current beacon detector is designed for a bright synthetic target and is not a general object-recognition model.
4. The Kalman model assumes approximately constant velocity over short intervals.
5. PID gains are fixed rather than learned or automatically scheduled.
6. The submitted repository does not contain a prebuilt standalone executable; Python execution is required unless the source is packaged separately.
7. The current source does not provide a multi-target association/handoff framework.
8. Atmospheric effects are synthetic image transformations, not a physical atmospheric propagation model.

These limitations do not prevent the prototype from demonstrating the requested software coarse-alignment loop, but they define the boundary between the present demonstrator and a flight/field-ready PAT subsystem.

18 Future Enhancements

1. Add a learned beacon detector trained on representative optical imagery.
2. Introduce a 3D scene and physically meaningful camera projection.
3. Add extended/unscented Kalman filtering for nonlinear motion.
4. Implement multi-target association and target handoff.
5. Add adaptive or reinforcement-learning-assisted controller tuning.
6. Add scripted benchmark scenarios and repeatable Monte Carlo experiments.
7. Add hardware-in-the-loop interfaces for real pan/tilt controllers.
8. Package the application using PyInstaller for a true standalone executable.
9. Add live performance plots and experiment comparison.
10. Model optical propagation effects such as beam divergence, scintillation, turbulence, and realistic sensor characteristics.

19 Reproducibility and Deployment

19.1 Environment

A supported Python installation with the packages listed in `requirements.txt` is sufficient for the source implementation.

19.2 Installation

From the project directory:

```
pip install -r requirements.txt
```

19.3 Execution

Launch the primary dashboard:

```
python dashboard.py
```

The headless acquisition verification can be run with:

```
python step4_edge_test.py
```

19.4 Output Files

When benchmark export is triggered, the logger creates:

- a timestamped telemetry CSV; and
- a timestamped performance summary JSON

inside the `logs/` directory.

20 Deliverable Readiness

Table 3: SIH deliverable readiness

Deliverable	Status	Notes
Software application	Source prototype ready	<code>dashboard.py</code> is the primary desktop application. Standalone packaging is a final deployment step.
Source code	Ready	Modular Python files with documented responsibilities.
Technical report	This document	Covers problem, architecture, algorithms, testing, performance, limitations and future work.

Deliverable	Status	Notes
User manual	Companion document	Covers setup, operation, controls, troubleshooting and report export.
Performance log	Implemented	CSV and JSON generated by <code>logger.py</code> .
Demonstration video	Optional	A short demonstration can use the supplied dashboard and benchmark export.

21 Conclusion

Team 404 Not Found’s SIH26169 prototype provides a complete software closed loop for coarse virtual FSOC alignment: a moving optical beacon is generated in a configurable virtual environment, a virtual camera acquires it using spiral search, computer vision estimates its centroid, a Kalman filter stabilizes and predicts the target state, and a PID controller continuously steers the virtual camera to reduce image-plane error.

The implementation also includes controlled environmental and sensor disturbances, real-time telemetry, benchmark thresholds, CSV frame logging, and JSON session reporting. The supplied edge-case acquisition test was executed successfully, acquiring the configured difficult target in approximately 0.30 seconds against a 2-second criterion.

The project is intentionally presented as a software coarse-alignment prototype. It does not claim physical optical hardware performance or a trained neural-network model that is not present in the source. Its modular structure nevertheless provides a practical foundation for learned detection, more realistic 3D optical simulation, adaptive control, and hardware-in-the-loop validation in subsequent iterations.

A Repository File Reference

File	Purpose
<code>dashboard.py</code>	Primary Tkinter desktop application and integrated closed-loop dashboard.
<code>step2_camera.py</code>	Virtual scene, camera viewport, target motion and disturbance engine.
<code>step4_search.py</code>	Archimedean spiral search and target visibility helper.
<code>step5_tracker.py</code>	Standalone/reference tracking implementation and OpenCV controls.
<code>step4_dashboard.py</code>	Earlier acquisition dashboard.
<code>step4_edge_test.py</code>	Headless edge-case acquisition test.
<code>logger.py</code>	CSV telemetry and JSON performance summary generation.
<code>requirements.txt</code>	Python dependencies.
<code>README.md</code>	Project overview and execution instructions.

B Key Parameters

Parameter	Implemented value/range
World size	2000×2000 px
Camera viewport	640×480 px
Reference FOV	4.0 degrees
Reference scale	160 px/degree
Beacon size	10 px default
Nominal FPS	30
Tracking error threshold	10 px
Acquisition threshold	2.0 s
Lost-frame threshold	15 frames
Pan speed	5.0–10.0 deg/s in GUI
Jitter	0–20 px/frame
Platform drift	0–20 px/frame
Atmosphere	clear/haze/fog/rain/low-light
Noise	none/salt-pepper/Gaussian/Poisson
PID gains	$K_p = 0.58$, $K_i = 0.035$, $K_d = 0.11$